Dijkstra's Shortest Path Algorithm

Understanding shortest paths in weighted graphs

Name: Sahil Kumar

PRN: 25070521280

Generated using: Canva



What is Dijkstra's Algorithm?

Finds shortest paths from one starting vertex to all others in a weighted graph with **non-negative** edge weights.

Initialize distances: start node is 0, others are infinity

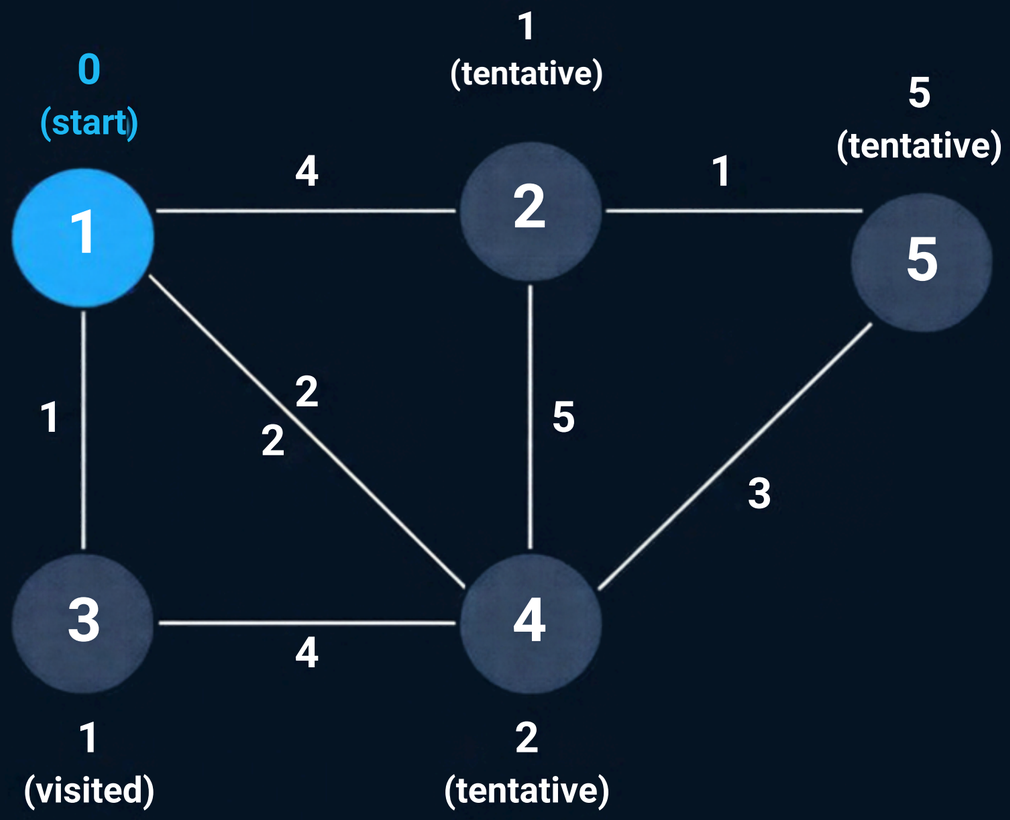
Select the unvisited node with the smallest distance

Update the distances to its neighbors

Mark the node visited and repeat

Continue until all nodes have been processed

Requires non-negative edge weights.



Pseudocode Overview

```
function Dijkstra(G, source):
for each vertex v in G:
dist[v] = ∞
prev[v] = null
dist[source] = 0
Q = priority queue of all vertices

Q is not empty:
u = extract-min(Q)
for each neighbor v of u:
alt = dist[u] + weight(u, v)
if alt < dist[v]:
dist[v] = alt
prev[v] = u
decrease-key(Q, v, alt)

return dist, prev
```

01 Initialize distances and track previous nodes

02 Use a priority queue to select the closest unvisited node

03 Relax edges and update distances when shorter paths are found

04 Repeat until the queue is empty

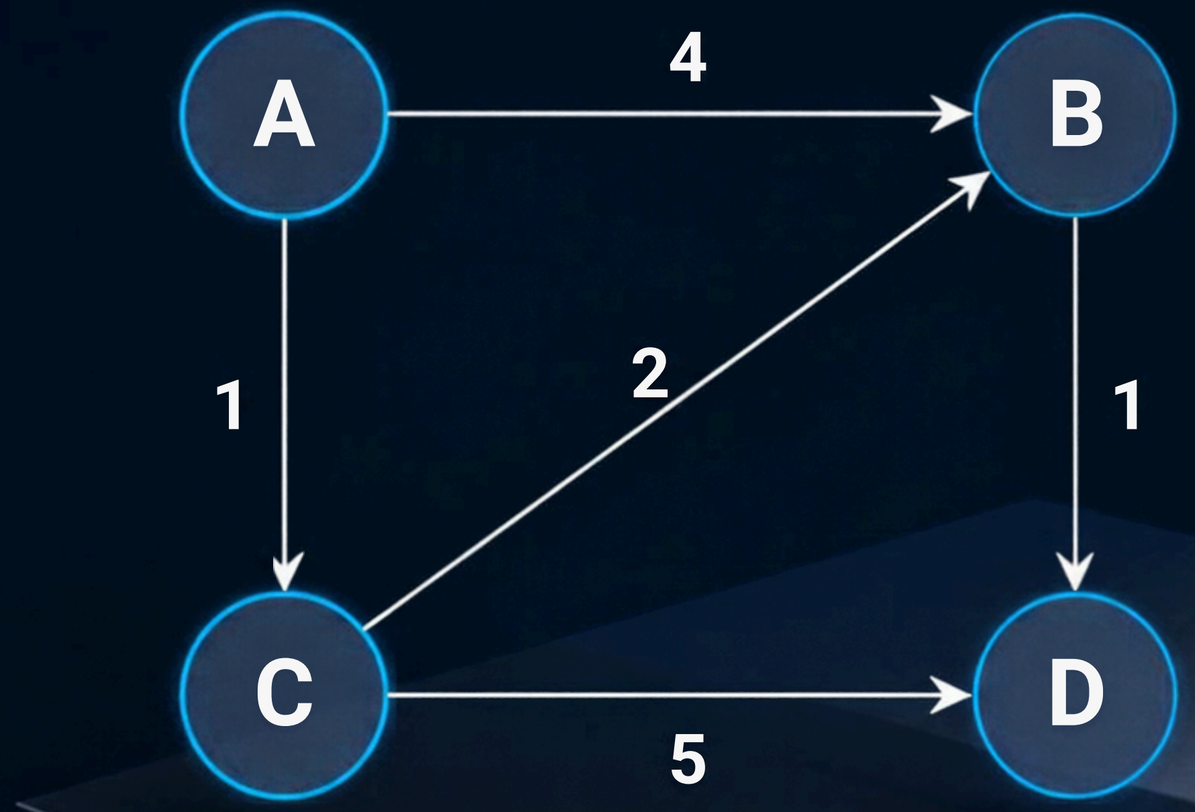
05 Return shortest distances and path information

Example Graph

Directed, weighted graph with four nodes

Edges show connection direction and cost

> Graph used for shortest path calculation starting at node A

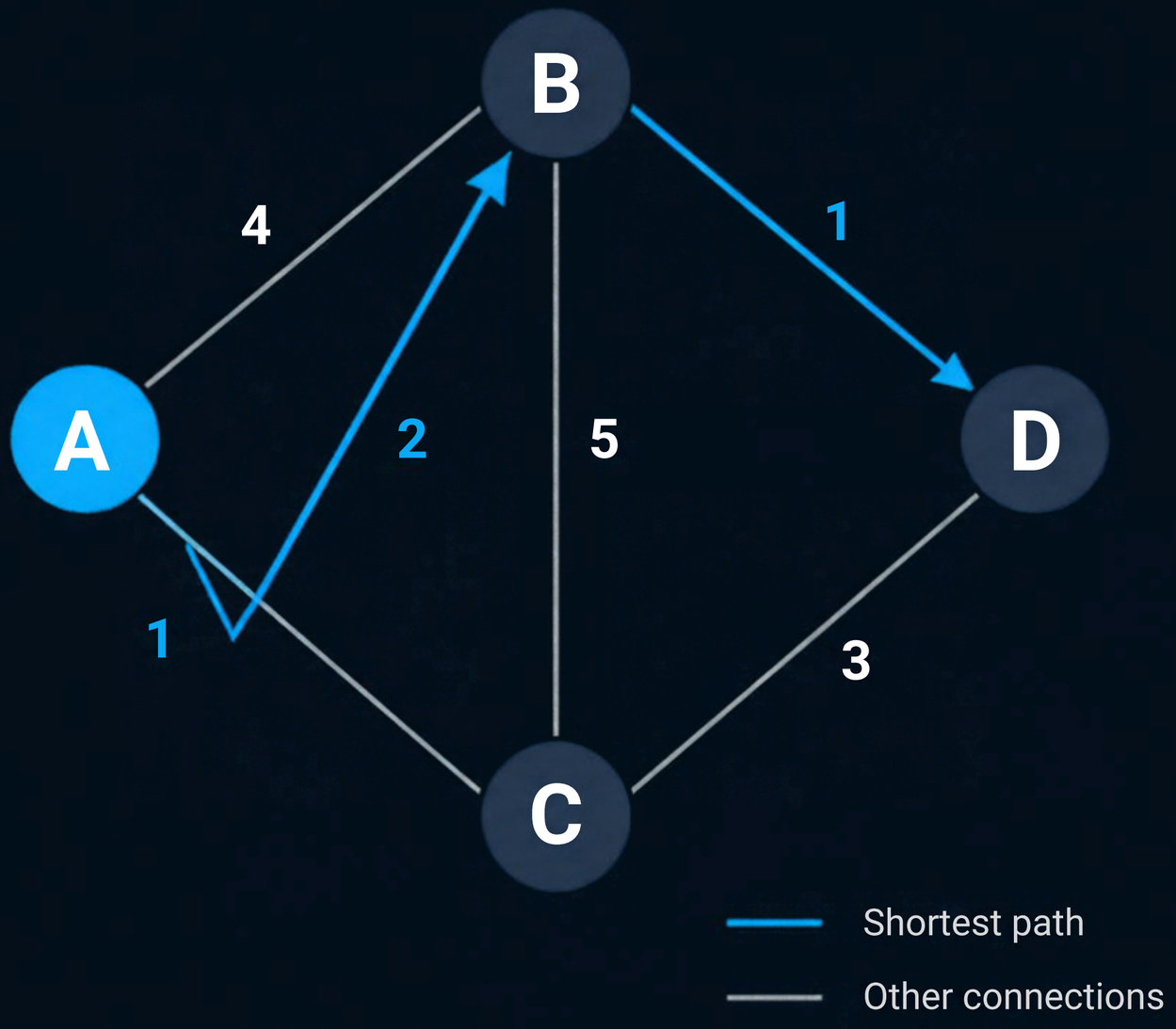


Shortest Paths from A

Using Dijkstra's algorithm, the minimum costs from A to all other nodes are:

1	A → C
3	A → C → B
4	A → C → B → D

Distances are improved by relaxing edges and updating neighbors whenever a lower cost is found. Once a node is visited, its shortest distance is final.



Python Implementation

Graph input uses an **adjacency list**
graph(u] = [(v, weight), ...]

Dijkstra's algorithm with **heapq**

Maintains shortest distances and **previous** nodes

Relax edges to update better paths

```python
import heapq

def dijkstra(graph, start):
    # initialization
    dist = {node: float('inf') for node in graph]
    prev = {node: None for node in graph}
    dist[start] = 0

    # priority queue
    pq = [(0, start)]
    heapq.heapify(pq)

    while pq:
      d, u = heapq.heappop(pq)
      if d > dist[u]:
      continue

        # relax edges
        for v, w in graph[u]:
          alt = d + w
          if alt < dist[v]:
            dist[v] = alt
            prev[v] = u
            heapq.heappush(pq, (alt, v))
  return dist, prev
```

Complexity of Dijkstra's Algorithm

Graph representation	Time complexity
Array/simple list	$O(V^2)$
Binary heap/priority queue	$O((V + E) \log V)$
Dense graph with heap	approx. $O(E \log V)$

V represents the number of **vertices** in the graph.

E represents the number of **edges** in the graph.

Important Limitation

Does not work correctly
with **negative edge weights**

Use **Bellman-Ford** algorithm
for graphs with negative edges



Key Takeaways

- Dijkstra finds shortest paths with non-negative weights
- Uses priority queue for efficient selection
- Time complexity depends on graph and data structure
- Not suitable for graphs with negative edges
- Python implementation is concise and clear



Thank you!

Questions and discussion
are welcome.

Sahil Kumar

PRN: **25070521280**

